

PARTE 1 – Realización de la auditoría

Encuentra y documenta las vulnerabilidades existentes en la aplicación.

Recuerda, que vuestro rol para realizar esta parte es de auditor de seguridad, no de desarrollador. A continuación, tenéis la ficha de auditoría a realizar para cada fallo detectado.

Principio	Línea y fichero en el que está ubicado el error:	Si fueras un atacante, ¿cómo lo utilizarías?	Indica el nivel de gravedad: alto, medio, bajo
Privilegio Minimo	Cada persona solo tiene solos los permisos q necesitas	El panel de administración se puede abrir escribiendo la opción 5, aunque el usuario no haya iniciado sesión. Aunque la aplicación no muestra esta opción en el menú, no comprueba si el usuario tiene permisos para entrar. Esto es un problema de seguridad, porque cualquier usuario podría acceder al panel y ver la lista de cuentas.	Alto
No confiar en la entrada	Lo que escribe el usuario puede ser malicioso, hay que tratarlo y comprobarlo antes de acabarlo	El panel de administración se ejecuta si se escribe aunque no se haya iniciado sesión. La aplicación oculta esa opción del menú, pero no comprueba los permisos al ejecutar la acción. Un usuario podría ver la lista de cuentas.	Medio

No confiar en la entrada (2)	La informacion que se muestra tambien puede revelar datos que no deberian ser publicos	El nombre de búsqueda se añade directamente a una consulta. Una entrada manipulada puede activar la inyección que simula el programa y mostrar el primer usuario	Bajo
Defensa en profundidad	No se debe depender de una única protección; se añaden varias barreras.	La aplicación crea siempre unas cuentas de prueba con contraseñas que ya se conocen. Además, estas cuentas aparecen cuando se inicia el programa. Esto puede ser un problema de seguridad, ya que una persona podría intentar entrar usando las cuentas de prueba, especialmente la de administrador. El problema está en, en las líneas 19 y 72.	Alto

Fallar de Forma segura	Si no se puede comprobar que una acción está permitida, se debe denegar	Se crean siempre cuentas de prueba con contraseñas conocidas y, además, se muestran al arrancar. Un atacante puede probar directamente las credenciales de administrador	alto
No inventar criptografía	Para proteger contraseñas y sesiones se deben usar mecanismos adecuados y probados.	Las contraseñas se guardan con MD5, que es demasiado rápido y no añade un valor aleatorio individual. Si alguien obtiene esos hashes, puede probar muchas contraseñas rápidamente. Para contraseñas se recomienda un algoritmo específico, como Argon2id, bcrypt o PBKDF2	Medio
No inventar criptografía (2)	Los tokens deben ser difíciles de adivinar y mantenerse privados.	El token se crea con Random tiene solo seis cifras y se muestra y guarda en texto plano. Alguien con acceso a la consola, los registros	Bajo

		o el archivo podría verlo. En esta aplicación el token no se usa para permitir acciones, así que el riesgo actual es limitado	
Defensa en profundidad	Los secretos deben protegerse en el código, al enviarse y al registrarse.	La clave del servicio está escrita directamente en el código y además se envía por HTTP dentro de la URL. También se muestra la URL por pantalla, por lo que la clave podría quedar expuesta. Si la clave fuera real, otra persona podría conseguirla mirando el código, el tráfico de red o los registros del programa. El problema está en, en las líneas 9 y 28.	medio
Fallar de forma segura	Los errores deben informar al usuario sin revelar detalles internos	Cuando ocurre un error en el programa, se muestra toda la información de la excepción. Esto puede ser un problema de seguridad, porque un usuario podría ver información sobre cómo está hecho el programa y usarla para encontrar posibles fallos. El problema está, en las líneas 58 y 61.	bajo

PARTE 2 – Corrección de los errores

Siguiendo los apuntes, corrige los errores detectados en el código. Modifica el código de la aplicación entregada.

Principio	Línea y fichero en el que está ubicado el error:	Error detectado	¿Cómo se ha corregido?
Privilegio mínimo	Program.cs, líneas 44 y 146.	El panel de administración se abría escribiendo la opción 5 aunque no se hubiera iniciado sesión. La aplicación solo ocultaba la opción en el menú, pero no comprobaba los permisos.	En PanelAdministracion() se comprueba que el usuario es administrador antes de mostrar nada; si no lo es, aparece «Acceso denegado». Además, el registro público solo crea cuentas con rol «jugador».
No confiar en la entrada	BaseDatosUsuarios.cs, líneas 30 y 43.	El nombre de búsqueda se añadía directamente a la consulta. Con una entrada manipulada como ' OR '1'='1 se activaba la inyección y se mostraba el primer usuario.	La entrada se trata como un dato y no como parte de la consulta: se valida con una lista blanca (solo letras, números y _, de 3 a 20 caracteres) y se compara el nombre exacto, igual que haría una consulta parametrizada.
No confiar en la entrada (2)	Program.cs, líneas 122 y 127.	Sin iniciar sesión se podían buscar usuarios y ver su rol, y así localizar la cuenta de administración.	Para buscar usuarios hay que haber iniciado sesión. El rol del usuario encontrado solo se muestra si quien busca es administrador.

Defensa en profundidad	Program.cs, líneas 19 y 72.	Se creaban siempre cuentas de prueba con contraseñas conocidas escritas en el código y, al arrancar, se avisaba de que existían.	Se elimina el aviso del arranque. Las cuentas de prueba solo se crean en la versión de depuración (#if DEBUG) y sus contraseñas se leen de variables de entorno, no del código.
Fallar de forma segura	Program.cs, línea 146.	La acción del panel no verificaba que la persona conectada fuera administradora; la comprobación solo se hacía al dibujar el menú.	La comprobación se hace justo antes de mostrar los datos y, si no se puede confirmar el rol, se deniega el acceso (denegar por defecto). La comprobación se centraliza en el método EsAdmin().
No inventar criptografía	AuthService.cs, línea 56.	Las contraseñas se guardaban con MD5, que es demasiado rápido y no añade un valor aleatorio (sal) para cada usuario.	Se sustituye MD5 por PBKDF2 con SHA-256, un algoritmo específico para contraseñas, con una sal aleatoria de 16 bytes por usuario y 600.000 iteraciones. La comparación de hashes se hace en tiempo constante.
No inventar criptografía (2)	AuthService.cs, líneas 47, 63 y 69. Program.cs, línea 143.	El token se creaba con Random, tenía solo seis cifras y se mostraba y guardaba en texto plano.	El token se genera con RandomNumberGenerator (32 bytes aleatorios), no se muestra nunca por pantalla y en el usuario solo se

			guarda su hash SHA-256.
Defensa en profundidad (2)	RedService.cs, líneas 9 y 28.	La clave del servicio estaba escrita en el código, se enviaba por HTTP dentro de la URL y además se imprimía en pantalla.	La clave se lee de una variable de entorno (GAME_API_KEY) y no se imprime. Solo se permiten conexiones HTTPS y la clave viaja en la cabecera Authorization, no en la URL.
Fallar de forma segura (2)	Program.cs, líneas 58 y 61.	Cuando ocurría una excepción se mostraba el error completo (ex.ToString()), con detalles de cómo está construido el programa.	El usuario solo ve un mensaje genérico («Ha ocurrido un error inesperado») y el detalle del error se envía únicamente a la traza de depuración.